

Relatório Parcial - Sprint 2

Número da Sprint: 2

Data da Entrega: 20/05/2026

Acesso ao sistema

URL: <https://web-production-1f724.up.railway.app>

Credenciais pré-cadastradas (senha padrão: `monitoria-app`):

- Willian Pessoa - willian.pessoa.cs@gmail.com (Admin)
- Gabriel - gabrielrb@ic.ufrj.br (Admin)
- Pedro - pedroaac@ic.ufrj.br (Admin)
- Gustavo - gustavopo@ic.ufrj.br (Admin)
- Joao Pedro - joaopmab@ic.ufrj.br (Admin)
- Product Owner - product-owner@monitoria-app.com.br (Admin)

Os usuarios sao criados automaticamente na inicializacao da aplicacao caso ainda nao existam.

Entregáveis da Sprint

- Cadastro das disciplinas - US06: admin cadastra disciplinas com nome, codigo e professor responsavel. Concluido.
- Cadastro das bolsas de monitoria - US07 + US08: professor indica aluno como monitor; admin aprova a indicacao, criando o vinculo de monitoria ativo. Concluido.
- Associacao monitor-disciplina - US07 + US08: o vinculo entre monitor e disciplina e estabelecido pela indicacao do professor e confirmado pela aprovacao do admin. Concluido.
- Listagem de monitorias ativas - US09: admin visualiza todas as monitorias ativas com disciplina, monitor e professor responsavel. Concluido.
- Publicacao em producao - TT06: aplicacao publicada no Railway com banco MySQL conectado e acessivel pela URL publica. Concluido.

Perguntas Gerais

Qual foi o tempo médio das reuniões Daily Stand-Up desta semana?

Houve uma Daily presencial em aula no dia 14/05, com duração aproximada de 10 minutos. Nos demais dias, o acompanhamento ocorreu de forma assíncrona via grupo do WhatsApp, com cada membro reportando o andamento das suas tarefas. Não houve outras reuniões formais de Daily ao longo da semana.

Houve levantamento de questões técnicas pela equipe antes de iniciarem o desenvolvimento das tarefas?

Não foram levantadas questões técnicas formais antes do início do desenvolvimento. As dúvidas foram resolvidas de forma assíncrona conforme surgiram durante a implementação.

Houve tarefas rejeitadas pelo Product Owner por erro de interpretação ou execução incorreta? Se sim, quais?

Não houve tarefas rejeitadas pelo Product Owner.

Bugs foram identificados na entrega final ou durante as revisões semanais? Se sim, quantos e quais?

Não foram identificados bugs funcionais nas histórias entregues. O QM identificou um problema de integração: a branch criada originalmente pelo responsável da US09 estava isolada e não havia sido integrada. Após revisão do QM, constatou-se que João Pedro havia implementado a funcionalidade dentro do trabalho da US07/US08 (PR #32), e a US09 estava integrada e funcional na branch principal.

Como se deu o levantamento de dúvidas e compartilhamento de conhecimento durante a sprint?

O compartilhamento de conhecimento ocorreu de forma assíncrona via WhatsApp e por revisão direta do código entre os membros.

Quantidade de Tarefas Planejadas: 4 (US06, US07, US08, US09) + 1 tarefa técnica (TT06)

Quantidade de Tarefas Concluídas (Done): 4 (US06, US07, US08, US09) + TT06

Outras Observações:

O Sprint Goal da Sprint 2 - "O admin consegue cadastrar disciplinas e o professor consegue indicar um monitor, que é aprovado pelo admin" - foi atingido. As quatro histórias do EP02 (US06, US07, US08, US09) foram implementadas e estão funcionando no ambiente de produção (Railway), além da tarefa técnica TT06 (publicação no Railway).

Durante a sprint, a Product Owner Bruna identificou uma lacuna no escopo de agendamento: o sistema precisava definir uma regra de cancelamento e cobrir o caso do monitor cancelar um agendamento confirmado. Foi criada a US "Monitor cancela agendamento confirmado" com regra de 6 horas de antecedência (must-have, issue #33), e a US14 existente (Aluno cancela agendamento) foi atualizada com a mesma regra.

Perguntas QScrum

Flags foram inseridas pelo Quality Manager esta semana?

Sim. Foram inseridas 4 flags.

- Quantidade: 4
 - Motivos:
1. Sprint Tales da Sprint 2 não havia sido preenchido ao término do desenvolvimento
 2. Lógica técnica não havia sido formalmente repassada ao QM
 3. US09 entrou em desenvolvimento sem critérios de aceitação definidos (falha no DoR)
 4. Branch da US09 não foi integrada à branch principal antes do encerramento da sprint
- Tempo entre inserção e resolução: Todas as 4 flags foram resolvidas ao encerramento da sprint. As flags de documentação (Sprint Tales e lógica técnica) foram resolvidas durante o fechamento. As flags da US09 foram resolvidas após revisão do QM que confirmou a implementação já integrada via PR #32; os critérios de aceitação foram definidos retroativamente e validados.

As Histórias de Usuário foram escritas no formato Given-When-Then? Se sim, o formato auxiliou de a

Sim. As histórias US06, US07 e US08 têm critérios de aceitação no formato BDD (Given / When / Then) em `docs/criterios-de-aceitacao.md` e nas respectivas issues do GitHub. O formato auxiliou a identificação de cenários de borda - em particular o cenário de "professor inválido" em US06 e o cenário de "aluno já monitor em outra disciplina" em US08, que poderiam ter sido ignorados sem a formalização dos critérios.

O Quality Manager validou as tarefas através do checklist do Definition of Ready?

Sim. O QM aplicou o checklist do DoR nas tarefas. A flag referente à US09 decorreu exatamente dessa validação - a história não possuía critérios de aceitação e não deveria ter entrado em desenvolvimento. Os critérios foram definidos retroativamente pelo QM. Após revisão, a implementação foi confirmada como integrada e a história foi fechada como Done.

Artefatos

- [x] Sprint Tales atualizado - docs/sprint-tales/sprint-2.md
 - [x] Explicação da lógica técnica repassada ao Quality Manager
 - [x] Sprint Backlog atualizado - issues #11, #12, #13, #14 no GitHub Projects
 - [x] Evidências da Sprint Review - US06, US07 e US08 testadas e funcionando no Railway
 - [x] Board da Retrospectiva Sprint 2 - docs/retrospectivas/sprint-2.md
 - [x] Relatório parcial - este documento
-

Sprint Tales - Sprint 2

Sprint: 2 - Cadastro do Domínio Principal

Período: 14/05/2026 - 20/05/2026

QM: Willian Gomes Pessoa

US06 - Admin cadastra disciplinas

Prioridade: Must. Responsavel: Gustavo. Alterado depois: Nao. Tempo suficiente: Sim.

Solucao adotada: CRUD de disciplinas com nome, codigo e professor responsavel. Validacao de codigo unico e de papel do professor. Operacoes de ativar/desativar e matricula de alunos em lote (por email) ou individual.

Arquivos alterados: `backend/disciplinas/repository.py`, `routes.py`, `service.py`, `frontend/templates/disciplinas/index.html`.

Impacto: Gestao de disciplinas pelo admin - base do EP02.

US07 - Professor indica aluno como monitor

Prioridade: Must. Responsavel: Joao Pedro. Alterado depois: Nao. Tempo suficiente: Sim.

Solucao adotada: Tela `GET/POST /monitorias/indicar` para professor selecionar uma das suas disciplinas e um aluno ativo. Validacao de propriedade da disciplina e de papel do aluno. Indicacao criada com status `PENDENTE\_APROVACAO`. Professor acompanha suas indicacoes na mesma tela.

Arquivos alterados: `backend/monitorias/routes.py`, `service.py`, `repository.py`, `frontend/templates/monitorias/indicar.html`.

Impacto: Fluxo de indicacao de monitores.

US08 - Admin aprova ou rejeita indicacao de monitor

Prioridade: Must. Responsavel: Pedro. Alterado depois: Nao. Tempo suficiente: Sim.

Solucao adotada: `GET /monitorias/pendentes` lista indicacoes aguardando aprovacao. `POST /monitorias/<id>/aprovar` ativa o vinculo; `POST /monitorias/<id>/rejeitar` registra o motivo. Validacao: aluno ja com monitoria ativa em qualquer disciplina e bloqueado na aprovacao.

Arquivos alterados: `backend/monitorias/routes.py`, `service.py`, `repository.py`, `frontend/templates/monitorias/pending.html`, `docs/adr/0006-admin-aprovacao-indicacao-monitor.md`.

Impacto: Controle de aprovacao de monitores pelo admin.

US09 - Admin lista monitorias ativas

Prioridade: Should. Responsavel: Joao Pedro. Alterado depois: Nao. Tempo suficiente: Sim.

Solucao adotada: Funcao `list\_active\_monitorias()` retorna todos os vinculos com `status = 'ATIVO'`. Exibida como tabela na tela de admin de disciplinas. Mensagem de vazio quando nao ha registros.

Arquivos alterados: `backend/monitorias/repository.py`, `service.py`,
`backend/disciplinas/routes.py`, `frontend/templates/disciplinas/index.html`.
Impacto: Visibilidade das monitorias ativas para o admin - completa EP02.

TT06 - Publicacao no Railway

Prioridade: Must. Responsavel: Willian. Alterado depois: Nao. Tempo suficiente: Sim.
Solucao adotada: Publicacao da aplicacao no Railway. Configuracao do `Procfile` (gunicorn),
`railpack.json` e `requirements.txt` na raiz. `config.py` adaptado para aceitar as variaveis de
ambiente do plugin MySQL do Railway. Schema e seed de admins aplicados automaticamente
na inicializacao.

Arquivos alterados: `Procfile`, `railpack.json`, `requirements.txt`, `backend/config.py`,
`backend/db/connection.py`.

Impacto: Aplicacao acessivel para as POs sem depender de ambiente local.

Explicação da Lógica Técnica - Sprint 2

Elaborado por: Desenvolvedores da Sprint 2

Destinado a: Willian Gomes Pessoa (Quality Manager)

Sprint: 2 - Cadastro do Domínio Principal (14/05 a 20/05/2026)

Visão geral dos módulos adicionados

A Sprint 2 expandiu a aplicação com dois novos blueprints ativos: `disciplinas` e `monitorias`. Ambos seguem o mesmo padrão da Sprint 1 (routes → service → repository), com queries SQL diretas e sem ORM.

...

browser

- └─ Flask (app.py)
- └─ auth/ ← Sprint 1
- └─ usuarios/ ← Sprint 1
- └─ disciplinas/ ← Sprint 2 (US06)
 - └─ routes.py (endpoints HTTP)
 - └─ service.py (regras de negócio)
 - └─ repository.py (queries SQL)
- └─ monitorias/ ← Sprint 2 (US07, US08)
 - └─ routes.py
 - └─ service.py
 - └─ repository.py

...

US06 - Cadastro de disciplinas

Fluxo principal (admin cadastra disciplina):

1. Admin preenche código, nome e seleciona professor responsável
2. Backend valida: código não pode ser duplicado; professor_id deve pertencer a um usuário com papel `PROFESSOR`
3. Disciplina criada com status `ATIVA`
4. Admin pode editar, desativar/reactivar e matricular alunos na disciplina

Matrícula de alunos:

- Individual: admin seleciona um aluno pelo dropdown
- Em lote: admin cola lista de emails separados por vírgula ou quebra de linha; o sistema ignora emails não encontrados e reporta quantos foram adicionados

Endpoints:

...

GET/POST /disciplinas/ (admin only)
POST /disciplinas/<id>/editar (admin only)
POST /disciplinas/<id>/desativar (admin only)
POST /disciplinas/<id>/ativar (admin only)
POST /disciplinas/<id>/matricular (admin only)
GET /disciplinas/<id>/alunos (admin e professor da disciplina)
POST /disciplinas/<id>/alunos/adicionar (admin only)
POST /disciplinas/<id>/alunos/remover (admin only)

...

Pontos de atenção para o QM:

- Professor só vê alunos da disciplina dele na rota /alunos - acesso verificado por session["user_id"]
- A matrícula em lote silencia erros de email não encontrado - comportamento intencional para não bloquear importações parciais

US07 - Professor indica aluno como monitor

Fluxo de indicação:

1. Professor acessa `GET /monitorias/indicar`

2. Sistema exibe apenas as disciplinas do professor logado (filtradas por `professor\_id = session["user\_id"]`)
3. Professor seleciona disciplina e aluno e confirma
4. Backend valida: disciplina pertence ao professor (verificação server-side); aluno tem papel `ALUNO`
5. Vínculo criado com status `PENDENTE\_APROVACAO`
6. Professor vê o histórico das suas indicações na mesma tela

Endpoints:

...

GET/POST /monitorias/indicar (professor only)

...

Pontos de atenção para o QM:

- A lista de alunos exibe todos os alunos ativos do sistema, não só os matriculados na disciplina - limitação conhecida, escopo da Sprint 3
- A validação de propriedade da disciplina é feita no backend, não apenas no frontend - um professor não consegue indicar para disciplina de outro mesmo forjando o form

US08 - Admin aprova ou rejeita indicação de monitor

Fluxo de aprovação:

1. Admin acessa `GET /monitorias/pendentes` - lista todas as indicações com status `PENDENTE\_APROVACAO`
2. Para aprovar: `POST /monitorias/<id>/aprovar`
 - Verifica se o aluno já possui monitoria ativa em qualquer disciplina (constraint de unicidade)
 - Se sim: rejeita com erro `ALUNO\_JA\_MONITOR`
 - Se não: atualiza status para `ATIVO`
3. Para rejeitar: `POST /monitorias/<id>/rejeitar`
 - Registra motivo informado pelo admin
 - Atualiza status para `REJEITADO`
 - Indicação sai da fila de pendentes

Endpoints:

...

GET /monitorias/pendentes (admin only)

POST /monitorias/<id>/aprovar (admin only)

POST /monitorias/<id>/rejeitar (admin only)

...

Decisão de design registrada em ADR-0006:

A validação de "aluno já monitor" é feita no momento da aprovação (não da indicação) para permitir que o professor indique sem precisar saber o estado atual de cada aluno. A responsabilidade de controle fica no admin.

Pontos de atenção para o QM:

- Indicação com status REJEITADO ou ATIVO não aparece mais na fila de pendentes
- Um aluno pode ter indicações pendentes em múltiplas disciplinas simultaneamente - apenas a primeira aprovada é aceita; as demais falham com ALUNO_JA_MONITOR

Migração de banco - Sprint 2

Duas alterações no schema em relação à Sprint 1:

```
```sql
-- Coluna adicionada via migration em connection.py
ALTER TABLE disciplinas ADD COLUMN status ENUM('ATIVA','INATIVA') NOT NULL
DEFAULT 'ATIVA';

-- Nova tabela para matrícula de alunos em disciplinas
CREATE TABLE IF NOT EXISTS disciplina_alunos (
 id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 disciplina_id BIGINT UNSIGNED NOT NULL,
 aluno_id BIGINT UNSIGNED NOT NULL,
 ...
 CONSTRAINT uq_disciplina_aluno UNIQUE (disciplina_id, aluno_id)
);
```
```

As migrations são aplicadas automaticamente na inicialização via `\_apply\_migrations()` em `connection.py`, de forma idempotente.

TT06 - Publicação no Railway

Problema: As POs precisam acessar o sistema sem depender do ambiente local de nenhum dev.

Solução: Publicação no Railway, plataforma de nuvem com detecção automática de Python e plugin MySQL nativo.

Como funciona:

1. O Railway detecta o projeto via `railpack.json` (provider Python) e executa o comando definido no `Procfile`:

...

```
web: gunicorn --chdir backend "app:create_app()" --bind 0.0.0.0:$PORT
```

...

2. O plugin MySQL do Railway injeta as variáveis de conexão com nomes diferentes dos padrões locais (`MYSQLHOST`, `MYSQLPORT`, `MYSQLUSER`, `MYSQLPASSWORD`, `MYSQLDATABASE`). O `backend/config.py` foi adaptado para aceitar ambos os formatos sem quebrar o ambiente local.

3. Na inicialização (`init\_db()` em `connection.py`), a aplicação aplica o schema e as migrations automaticamente - sem necessidade de rodar scripts manuais no banco de produção.

4. O mesmo `init\_db()` executa `\_seed\_admins()`, que cria os usuários pré-cadastrados do time e da PO caso ainda não existam.

Arquivos alterados:

- Procfile - comando de start para o gunicorn
- railpack.json - provider e start command para o Railway
- requirements.txt - movido para a raiz (Railway auto-detect)
- backend/config.py - suporte às variáveis do plugin MySQL do Railway
- backend/db/connection.py - `\_apply\_schema`, `\_apply\_migrations` e `\_seed\_admins` chamados em `init\_db()`